

COMPILER DESIGN LAB PROGRAMS

Programs with Sample Input and Output

1. Lexical Analyzer

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char str[])
{
    char keywords[8][10] =
    {"int", "float", "char", "if", "else", "for", "while", "return"};
    int i;
    for(i = 0; i < 8; i++)
        if(strcmp(str, keywords[i]) == 0)
            return 1;
    return 0;
}

int main()
{
    char str[100], token[20];
    int i = 0, j;

    printf("Enter a statement: ");
    fgets(str, sizeof(str), stdin);

    while(str[i] != '\\0')
    {
        if(isspace(str[i])) i++;
        else if(isalpha(str[i]))
        {
            j = 0;
            while(isalnum(str[i])) token[j++] = str[i++];
            token[j] = '\\0';

            if(isKeyword(token))
                printf("%s : Keyword\\n", token);
            else
                printf("%s : Identifier\\n", token);
        }
        else if(isdigit(str[i]))
        {
            j = 0;
            while(isdigit(str[i])) token[j++] = str[i++];
            token[j] = '\\0';
            printf("%s : Number\\n", token);
        }
        else
        {
            printf("%c : Operator/Special Symbol\\n", str[i]);
            i++;
        }
    }
    return 0;
}
```

Sample Output:

```
Enter a statement: int a = 10 + b;
int : Keyword
```

a : Identifier
= : Operator/Special Symbol
10 : Number
+ : Operator/Special Symbol
b : Identifier
; : Operator/Special Symbol

2. DFA Simulation

Program:

```
#include <stdio.h>

int main()
{
    char str[50];
    int state = 0, i;

    printf("Enter a binary string: ");
    scanf("%s", str);

    for(i = 0; str[i] != '\\0'; i++)
    {
        if(str[i] == '0')
            state = 0;
        else if(str[i] == '1')
            state = 1;
    }

    if(state == 1)
        printf("String Accepted\\n");
    else
        printf("String Rejected\\n");

    return 0;
}
```

Sample Output:

```
Enter a binary string: 10101
String Accepted
```

3. Three Address Code

Program:

```
#include <stdio.h>

int main()
{
    printf("Expression: a + b * c\\n\\n");
    printf("Three Address Code:\\n");
    printf("t1 = b * c\\n");
    printf("t2 = a + t1\\n");
    return 0;
}
```

Sample Output:

Expression: a + b * c

Three Address Code:

t1 = b * c

t2 = a + t1

4. Quadruples

Program:

```
#include <stdio.h>

int main()
{
    printf("Expression: a + b * c\\n\\n");
    printf("Operator\\tArg1\\tArg2\\tResult\\n");
    printf("-----\\n");
    printf("*\\t\\tb\\tc\\tt1\\n");
    printf("+\\t\\ta\\tt1\\tt2\\n");
    return 0;
}
```

Sample Output:

Expression: a + b * c

Operator	Arg1	Arg2	Result
*	b	c	t1
+	a	t1	t2

5. Triples

Program:

```
#include <stdio.h>

int main()
{
    printf("Expression: a + b * c\\n\\n");
    printf("No.\\t0operator\\tArg1\\tArg2\\n");
    printf("-----\\n");
    printf("0\\t*\\t\\tb\\tc\\n");
    printf("1\\t+\\t\\ta\\t(0)\\n");
    return 0;
}
```

Sample Output:

Expression: a + b * c

No.	Operator	Arg1	Arg2
0	*	b	c
1	+	a	(0)

6. Constant Folding

Program:

```
#include <stdio.h>

int main()
{
    int result;
    printf("Before Optimization:\\n");
    printf("x = 10 + 20\\n");
    result = 10 + 20;
    printf("\\nAfter Constant Folding:\\n");
    printf("x = %d\\n", result);
    return 0;
}
```

Sample Output:

```
Before Optimization:
x = 10 + 20
```

```
After Constant Folding:
x = 30
```

7. Common Subexpression Elimination

Program:

```
#include <stdio.h>

int main()
{
    printf("Before Optimization:\\n");
    printf("t1 = a + b\\n");
    printf("t2 = a + b\\n");
    printf("t3 = t1 * c\\n");

    printf("\\nAfter Optimization:\\n");
    printf("t1 = a + b\\n");
    printf("t2 = t1\\n");
    printf("t3 = t1 * c\\n");
    return 0;
}
```

Sample Output:

Before Optimization:

```
t1 = a + b
t2 = a + b
t3 = t1 * c
```

After Optimization:

```
t1 = a + b
t2 = t1
t3 = t1 * c
```


8. Dead Code Elimination

Program:

```
#include <stdio.h>

int main()
{
    printf("Before Optimization:\\n");
    printf("a = 10\\n");
    printf("b = 20\\n");
    printf("c = a + b\\n");
    printf("d = 50\\n");

    printf("\\nAfter Optimization:\\n");
    printf("a = 10\\n");
    printf("b = 20\\n");
    printf("c = a + b\\n");
    return 0;
}
```

Sample Output:

Before Optimization:

```
a = 10
b = 20
c = a + b
d = 50
```

After Optimization:

```
a = 10
b = 20
c = a + b
```

9. Target Code Generation

Program:

```
#include <stdio.h>

int main()
{
    printf("Three Address Code:\\n");
    printf("t1 = b * c\\n");
    printf("t2 = a + t1\\n");

    printf("\\nTarget Code:\\n");
    printf("MOV R1, b\\n");
    printf("MUL R1, c\\n");
    printf("MOV R2, a\\n");
    printf("ADD R2, R1\\n");
    return 0;
}
```

Sample Output:

Three Address Code:

t1 = b * c
t2 = a + t1

Target Code:

MOV R1, b
MUL R1, c
MOV R2, a
ADD R2, R1

10. NFA to DFA

Program:

```
#include <stdio.h>

int main()
{
    printf("NFA to DFA Conversion\\n\\n");

    printf("NFA Transition Table:\\n");
    printf("State\\t0\\t1\\n");
    printf("q0\\tq0\\tq0,q1\\n");
    printf("q1\\t-\\tq1\\n");

    printf("\\nEquivalent DFA States:\\n");
    printf("{q0}\\n");
    printf("{q0,q1}\\n");

    printf("\\nDFA Transition Table:\\n");
    printf("State\\t\\t0\\t\\t1\\n");
    printf("{q0}\\t\\t{q0}\\t\\t{q0,q1}\\n");
    printf("{q0,q1}\\t{q0}\\t{q0,q1}\\n");
    return 0;
}
```

Sample Output:

NFA to DFA Conversion

NFA Transition Table:

State	0	1
q0	q0	q0, q1
q1	-	q1

Equivalent DFA States:

{q0}
{q0, q1}

DFA Transition Table:

State	0	1
{q0}	{q0}	{q0, q1}
{q0, q1}	{q0}	{q0, q1}

11. Shift-Reduce Parser (Demonstration)

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char stack[50], input[50];
    int top = -1, i = 0;

    printf("Enter input string: ");
    scanf("%s", input);

    printf("\nStack\t\tInput\t\tAction\n");

    while(input[i] != '\\0')
    {
        stack[++top] = input[i++];
        stack[top + 1] = '\\0';
        printf("%s\\t%s\\tShift\\n", stack, input + i);

        if(top >= 2 && stack[top] == 'd' && stack[top-1] == 'd')
        {
            stack[top-1] = 'E';
            top--;
            stack[top+1] = '\\0';
            printf("%s\\t%s\\tReduce\\n", stack, input + i);
        }
    }

    printf("\nInput accepted using Shift-Reduce parsing.\\n");
    return 0;
}
```

Sample Output:

Enter input string: dd

Stack	Input	Action
d	d	Shift
dd		Shift
E		Reduce

Input accepted using Shift-Reduce parsing.

12. Simple FIRST Demonstration

Program:

```
#include <stdio.h>

int main()
{
    printf("Grammar:\\n");
    printf("E -> TR\\n");
    printf("R -> +TR | e\\n");

    printf("\\nFIRST(E) contains FIRST(T).\\n");
    printf("For the complete grammar, FIRST/FOLLOW should be calculated using a parser algorithm.\\n");

    return 0;
}
```

Sample Output:

Grammar:

E -> TR

R -> +TR | e

FIRST(E) contains FIRST(T).

For the complete grammar, FIRST/FOLLOW should be calculated using a parser algorithm.